

Joshua Rothe; Narek Abrahamian
Professor Khavari
ECE425L
Fall 2015
Lab Report 9
Group #4
12 Nov 2015

Objective

The purpose of this lab is to develop programs that display text on the LPC2148's LCD display. This display is equipped with a 2 x 16 character LCD. We were to write a message that was both stationary, and would scroll/rotate as needed.

Procedure

We first prepared the code outside of the lab environment, simulating with the debugger and compiling to ensure no errors.

Task 1: Write the following subroutines for LCD display and group them in one program file named as *lcd_subs.s*:

- ✓ Wait_10 μ s or Wait_1 μ s which can produce multiples of 10 μ s (or 1 μ s) time delay.
- ✓ LCD_pins which configures all port pins of the LPC2148 for LCD interface
- ✓ LCD_init which follows the initialization sequence to wake up LCD and configure its functions
- ✓ LCD_cmd which sends a command to the LCD
- ✓ LCD_char which sends a character to the LCD
- ✓ LCD_string which display a string starting from a certain location on the LCD.
- ✓ LCD_clear which clears the display and set cursor home.

This task was completed within several subassemblies which were called by the main assembly file, which was eventually named task3.s.

For the wait file, we programmed a delay subroutine that would be able to wait for any value we chose, by setting a constant in r0 before calling the subroutine. This allowed for reusing the same delay file with only one extra line of code per delay, instead of multiple subroutines to delay different values.

For LCD_pins, we followed the instructions given in the lab, configuring pins in PINSEL1 and 2 as GPIO, setting pins 0.22 and 0.30 as outputs, and setting pins 1.16-1.25 as outputs. We also added an additional command to turn on the LCD backlight.

For LCD_init, we followed the guidelines given in the lab instruction:

- (1) E = RS = RW = 0, and wait for at least 15ms.
- (2) Send command 0x30 to the pins D[7:0] of the LCD, and wait for at least 4.1ms.
- (3) Send command 0x30 and wait for at least 100μs.
- (4) Send command 0x30 and wait for at least 4.1ms.
- (5) Send command 0x38 to the LCD.
- (6) Send command 0x0C to the LCD.
- (7) Send command 0x01 to the LCD.
- (8) Send command 0x06 to the LCD.

We used the wait subroutine as a convenient way to pause for these variable amounts of time, and used the LCD_cmd subroutine to execute each step.

For LCD_cmd, we used the following guidelines:

- (1) D[7:0] = command (*passed from the calling program*).
- (2) RS = 0, R/W = 0, and E = 0 ; and wait for at least 6us.
- (3) E = 1 ; and wait for at least 6us.
- (4) E = 0; and wait for at least 40us.

After the above steps, we wrote the command to memory using LCD_DATA and bits 23-16 (which are D[0] thru D[7])

For LCD_char, we used the following, and similarly ended the program with a write to LCD_DATA:

- (1) D[7:0] = character (*passed from the calling program*).
- (2) RS = 1, R/W = 0, and E = 0; and wait for at least .
- (3) E = 1 ; and wait for at least .
- (4) E = 0; and wait for at least .

For LCD_string, we utilized the LCD memory map and register values from the top level assembly code. With the location of the first character and the start location of the string brought in from the top level, we were able to use LCD_cmd and LCD_char to set the position and begin inputting the text, stopping upon reaching the terminating “_” character.

Task 2: Write a program code to display two character strings on the first and second line of the LCD, respectively, by calling the subroutines written in *lcd_subs.s*. Note that the strings to display need to be stored in the memory.

For this task, we used LCD_subs and had DCB reserve strings for the code, and had them execute using the prepared subroutine LCD_string (after setting the pins and initializing the board). We merely needed a r0 value to point to the first character for the line, and r1 to trigger either the first or second line, and the subroutines took care of the rest.

Task 3: Generate a rotating display on the first line of the LCD; more specifically, continually rotate a character string from right to left, and the first character will appear from the right again and continue rotating from right to left.

For this task, we added shifting loops to allow for a constant shifting of the text. First we shifted the text to the right 17 spaces (rightshiftloop), then we used a large left shift loop to simulate a slow shifting to the left of the text (with a delay so the shifting was slow enough to be readable). Finally we used a second right shift loop (rightshiftloop2) to bring the strings back to the beginning position. A continuous loop of the slow left shift and rightshiftloop2 allowed the text to be displayed continuously.

Data

Display	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13	14	15	16
Line 1	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	10	11	12	13	14	15	...
Line 2	40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F	50	51	52	53	54	55	...

Figure 2. 2x16 LCD memory map.

Table 1. Pin assignment of LCD module.

Pin number	Symbol	I/O	Function
1	GND	-	Power supply (GND)
2	VCC	-	Power supply (+5V)
3	VLCD	-	Contrast adjust
4	RS	Input	0 = Command input; 1 = Data input
5	R/W	Input	0 = Write to LCD; 1 = Read from LCD
6	E	Input	Enable signal
7	D0	Input/Output	Data bus line 0 (LSB)
8	D1	Input/Output	Data bus line 1
9	D2	Input/Output	Data bus line 2
10	D3	Input/Output	Data bus line 3
11	D4	Input/Output	Data bus line 4
12	D5	Input/Output	Data bus line 5
13	D6	Input/Output	Data bus line 6
14	D7	Input/Output	Data bus line 7 (MSB)

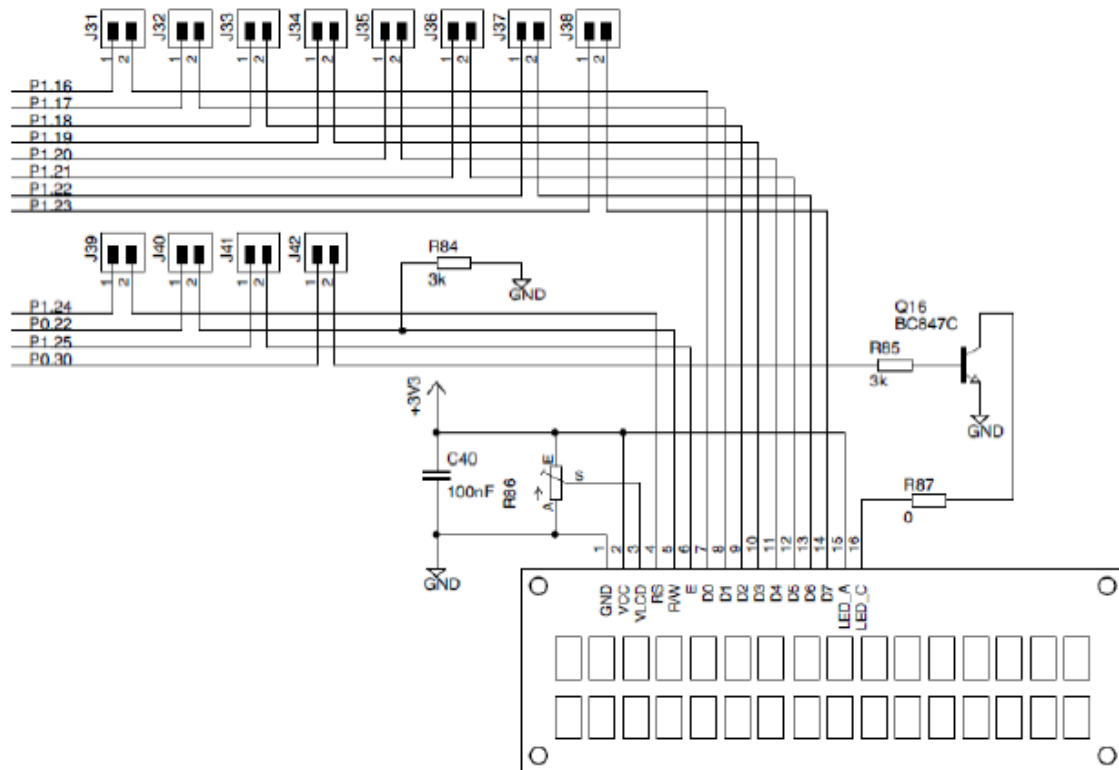


Figure 1. LPC 2148 Education Board Schematic: LCD.

Appendix: LCD Instruction Set

LCD instruction set												
Instruction	Code										Description	Execution time**
	RS	R/W	D7	D6	D5	D4	D3	D2	D1	D0		
Clear display	0	0	0	0	0	0	0	0	0	1	Clears display and returns cursor to the home position (address 0).	1.64mS
Cursor home	0	0	0	0	0	0	0	0	1	*	Returns cursor to home position (address 0). Also returns display being shifted to the original position. DDRAM contents remains unchanged.	1.64mS
Entry mode set	0	0	0	0	0	0	0	1	I/D	S	Sets cursor move direction (I/D), specifies to shift the display (S). These operations are performed during data read/write.	40μS
Display On/Off control	0	0	0	0	0	0	1	D	C	B	Sets On/Off of all display (D), cursor On/Off (C) and blink of cursor position character (B).	40μS
Cursor/display shift	0	0	0	0	0	1	S/C	R/L	*	*	Sets cursor-move or display-shift (S/C), shift direction (R/L). DDRAM contents remains unchanged.	40μS
Function set	0	0	0	0	1	DL	N	F	*	*	Sets interface data length (DL), number of display line (N) and character font(F).	40μS
Set CGRAM address	0	0	0	1	CGRAM address						Sets the CGRAM address. CGRAM data is sent or received after this setting.	40μS
Set DDRAM address	0	0	1	DDRAM address							Sets the DDRAM address. DDRAM data is sent or received after this setting.	40μS
Read busy-flag and	0	1	BF	DDRAM address							Reads Busy-flag (BF) indicating internal operation	0μS

address counter				is being performed and reads address counter contents.	
Write to CGRAM or DDRAM	1	0	write data	Writes data to CGRAM or DDRAM.	40 μ S
Read from CGRAM or DDRAM	1	1	read data	Reads data from CGRAM or DDRAM.	40 μ S

Notes:

- DDRAM = Display Data RAM.
- CGRAM = Character Generator RAM.
- DDRAM address corresponds to cursor position.
- Address Counter is used for both DDRAM and CGRAM.
- * = Don't care.
- ** = Based on Fosc = 250KHz.

Bit names		
Bit	Settings	
I/D	0 = Decrement cursor position	1 = Increment cursor position
S	0 = No display shift	1 = Display shift
D	0 = Display off	1 = Display on
C	0 = Cursor off	1 = Cursor on
B	0 = Cursor blink off	1 = Cursor blink on
S/C	0 = Move cursor	1 = Shift display
R/L	0 = Shift left	1 = Shift right
DL	0 = 4-bit interface	1 = 8-bit interface
N	0 = 1/8 or 1/11 Duty (1 line)	1 = 1/16 Duty (2 lines)
F	0 = 5x7 dots	1 = 5x10 dots
BF	0 = Can accept instruction	1 = Internal operation in progress

LCD_subs

```

1 ;Joshua Rothe, joshrothe@gmail.com
2 ;Narek Abrahamian, narek.abrahamian.287@my.csun.edu
3 ;Lab 9 part 2, Ver 0.02, last modified 5 Nov 2015
4 ;displays characters on the LCD display, shifting
5
6     IMPORT      Wait_10us
7     IMPORT      LCD_pins
8     IMPORT      LCD_init
9     IMPORT      LCD_cmd
10    IMPORT      LCD_char
11    IMPORT      LCD_string
12
13    GLOBAL      user_code
14    AREA        reset, CODE, Readonly
15
16    SRAM_BASE   EQU      0x40000400
17
18    user_code
19
20    LDR          sp, =SRAM_BASE
21
22    BL           LCD_pins      ;configures pins
23    BL           LCD_init      ;initialize board
24    LDR          r0, =char1
25    MOV          r1, #0        ;indicates first line
26    BL           LCD_string
27    LDR          r0, =char2
28    MOV          r1, #1        ;triggers second line
29    BL           LCD_string
30
31    ;wait at least 2500000us
32    LDR          r0, =500000 ;wait count
33    BL           Wait_10us
34
35    MOV          r2, #17      ;shift count
36    rightshiftloop ;shifts right initially 17 spaces, to start from right edge
37                                ;D4, D3, D2 high: D4 enables cursor/display shift,
38                                ;D3 sets display as shifting, D2 sets right dir
39    MOV          r0, #0x1C    ;D4, D3, D2 high:
40    BL           LCD_cmd
41    SUBS         r2, r2, #1
42    BNE          rightshiftloop
43
44    bigloop
45
46    MOV          r2, #34      ;shift count
47    leftshiftloop
48                                ;D4, D3 high, D2 low: D4 enables cursor/display shift,
49                                ;D3 sets display as shifting, D2 low sets left dir
50    MOV          r0, #0x18    ;

```

```

51          BL          LCD_cmd
52          LDR          r0, =85000      ;wait count to make LCD letters visible
53          BL          Wait_10us        ;as it shifts across display
54          SUBS          r2, r2, #1
55          BNE          leftshiftloop
56
57          MOV          r2, #34          ;shift count
58 rightshiftloop2      ;shifts right 34 spaces, resets location of strings to right edge
59          MOV          r0, #0x1C        ;see comments for previous right shift
60          BL          LCD_cmd
61          SUBS          r2, r2, #1
62          BNE          rightshiftloop2
63
64          B            bigloop
65
66 char1              DCB          ".1738. im like hey", "_ "
67 char2              DCB          "whats up hello", "_ "
68          ALIGN
69
70 stop              B            stop
71          END

```


LCD_pins

```
1 ;Joshua Rothe, joshrothe@gmail.com
2 ;Narek Abrahamian, narek.abrahamian.287@my.csun.edu
3 ;Lab 9 LCD_pins, Ver 0.02, last modified 5 Nov 2015
4 ;configures lcd pins
5
6             GLOBAL LCD_pins
7             AREA reset, CODE, Readonly
8
9 LCD_pins
10
11 PINSEL1     EQU     0xE002C004           ;pin function selection for port 0
12 PINSEL2     EQU     0xE002C014           ;pin function selection for port 1
13 IOODIR      EQU     0xE0028008           ;sets signal direction of port 0
14 IO1DIR      EQU     0xE0028018           ;as above, but for port 1
15 IOOPIN      EQU     0xE0028000
16 LCD_LIGHT   EQU     0x40000000
17
18             STMFD sp!, {r0-r2,lr}
19
20             LDR     r1, =PINSEL1         ;setting pins as GPIO
21             LDR     r0, [r1]             ;clear pins 0.22 and 0.30
22             LDR     r2, =0x30003000      ;clearing bits 13:12 and 29:28
23             BIC     r0, r0, r2           ;22-16=6, 2(6)+1:2(6)
24             STR     r0, [r1]             ;30-16=14, 2(14)+1:2(14)
25
26             LDR     r1, =PINSEL2
27             LDR     r0, [r1]
28             BIC     r0, r0, #0x00000008
29             STR     r0, [r1]             ;clearing bit 3
30
31             LDR     r1, =IOODIR
32             LDR     r0, =0x40400000      ;setting P0.22 P0.30 outputs
33             STR     r0, [r1]
34
35             LDR     r1, =IO1DIR
36             LDR     r0, =0x03FF0000      ;setting P0.16-P0.25 outputs
37             STR     r0, [r1]
38
39             LDR     r1, =IOOPIN           ;turning on LCD backlight
40             LDR     r0, [r1]
41             ORR     r0, r0, #LCD_LIGHT
42             STR     r0, [r1]
43
44             LDMFD sp!, {r0-r2,pc}
45
46             END
```

LCD_init

```
1 ;Joshua Rothe, joshrothe@gmail.com
2 ;Narek Abrahamian, narek.abrahamian.287@my.csun.edu
3 ;Lab 9 LCD_init, Ver 0.02, last modified 5 Nov 2015
4 ;initializes the display
5
6             GLOBAL LCD_init
7             AREA reset, CODE, Readonly
8
9 LCD_init
10
11             IMPORT LCD_cmd
12             IMPORT Wait_10us
13
14 IOOPIN      EQU      0xE0028000
15 IOIPIN      EQU      0xE0028004
16
17 LCD_DATA    EQU      0x00FF0000      ;P1.16 - P1.23 are D0 thru D7
18 LCD_RS      EQU      0x01000000      ;p1.24, 0 is command, 1 is text
19 LCD_E       EQU      0x02000000      ;p1.25, 1-0 tells processor to take data from bus
20 LCD_RW      EQU      0x00400000      ;P0.22, 0 is writing to LCD, 1 is querying LCD
21 LCD_LIGHT   EQU      0x40000000      ;P0.30
22
23             STMFDP sp!, {r0-r2,lr}
24
25             ;rs = 0, rw = 0, e = 0
26             LDR     r1, =IOIPIN
27             LDR     r0, [r1]
28             BIC     r0, #LCD_E
29             BIC     r0, #LCD_RS
30             STR     r0, [r1]
31
32             LDR     r1, =IOOPIN
33             LDR     r0, [r1]
34             BIC     r0, #LCD_RW
35             STR     r0, [r1]
36
37             ;wait at least 15000us
38             LDR     r0, =30000
39             BL      Wait_10us      ;wait
40
41             ;send command to pins
42             LDR     r0, =0x30      ;r0 is command for sub
43             BL      LCD_cmd
44
45             ;wait at least 4100us
46             LDR     r0, =8200
47             BL      Wait_10us      ;wait
48
49             ;send cmd 0x30
50             LDR     r0, =0x30
```

```

51      BL      LCD_cmd
52
53      ;wait at least 100us
54      MOV     r0, #200
55      BL      Wait_10us      ;wait
56
57      ;send cmd 0x30
58      LDR     r0, =0x30
59      BL      LCD_cmd
60
61      ;wait at least 4100us
62      LDR     r0, =8200
63      BL      Wait_10us      ;wait
64
65      ;send cmd 0x38
66      LDR     r0, =0x38
67      BL      LCD_cmd
68
69      ;send cmd 0x0C
70      LDR     r0, =0x0C
71      BL      LCD_cmd
72
73      ;send cmd 0x01
74      LDR     r0, =0x01
75      BL      LCD_cmd
76
77      ;send cmd 0x06
78      LDR     r0, =0x06
79      BL      LCD_cmd
80
81      LDR     r0, =0xFF
82      BL      LCD_cmd
83
84      LDMFD   sp!, {r0-r2,pc}
85
86      END

```

LCD_cmd

```
1 ;Joshua Rothe, joshrothe@gmail.com
2 ;Narek Abrahamian, narek.abrahamian.287@my.csun.edu
3 ;Lab 9 LCD_cmd, Ver 0.02, last modified 5 Nov 2015
4 ;enters a command into the lcd
5
6         GLOBAL LCD_cmd
7         AREA      reset, CODE, Readonly
8
9 LCD_cmd
10
11         IMPORT Wait_10us
12
13 IOOPIN    EQU      0xE0028000
14 IOIPIN    EQU      0xE0028010
15
16 LCD_DATA  EQU      0x00FF0000      ; P1.16 - P1.23
17 LCD_RS    EQU      0x01000000      ; p1.24, 0 is command, 1 is text
18 LCD_E     EQU      0x02000000      ; p1.25, 1-0 tells processor to take data from bus
19 LCD_RW    EQU      0x00400000      ; P0.22, 0 is writing to LCD, 1 is querying LCD
20 LCD_LIGHT EQU      0x40000000      ; P0.30
21
22         STMFD    sp!, {r0-r2,lr}
23
24         MOV      r2,r0      ;command is in r0
25
26         ;rs = 0, rw = 0, and e = 0
27         LDR      r1, =IOIPIN
28         LDR      r0, [r1]
29         BIC      r0, #LCD_E ;e make low
30         BIC      r0, #LCD_RS ;rs low
31         STR      r0, [r1]
32
33         LDR      r1, =IOOPIN
34         LDR      r0, [r1]
35         BIC      r0, #LCD_RW ;rw low
36         STR      r0, [r1]
37
38         ;wait at least 6us
39         MOV      r0, #20      ;6 us
40         BL       Wait_10us    ;wait
41
42         ;e = 1
43         LDR      r1, =IOIPIN
44         LDR      r0, [r1]
45         ORR      r0, #LCD_E ;e low again
46         STR      r0, [r1]
47
48         ;wait at least 6us
49         MOV      r0, #20
50         BL       Wait_10us    ;wait
```

```

51
52      ;writing command to memory
53      LDR      r1, =IO1PIN
54      LDR      r0, [r1]
55      BIC      r0, #LCD_DATA      ;clear bits 23-16
56      ORR      r0, r2, LSL #16 ;inputting actual command
57      STR      r0, [r1]
58
59      ;e = 0
60      LDR      r1, =IO1PIN
61      LDR      r0, [r1]
62      BIC      r0, #LCD_E      ;e = low
63      STR      r0, [r1]
64
65      ;wait at least 40us
66      MOV      r0, #80
67      BL       Wait_10us      ;wait
68
69      LDMFD     sp!, {r0-r2,pc}
70
71      END

```

LCD_char

```
1 ;Joshua Rothe, joshrothe@gmail.com
2 ;Narek Abrahamian, narek.abrahamian.287@my.csun.edu
3 ;Lab 9 LCD_char, Ver 0.02, last modified 5 Nov 2015
4 ;enters a character into the lcd
5
6             GLOBAL LCD_char
7             AREA reset, CODE, Readonly
8
9 LCD_char
10
11            IMPORT Wait_10us
12
13 IOOPIN      EQU      0xE0028000
14 IOIPIN      EQU      0xE0028010
15
16 LCD_DATA    EQU      0x00FF0000      ; P1.16 - P1.23
17 LCD_RS      EQU      0x01000000      ; p1.24, 0 is command, 1 is text
18 LCD_E       EQU      0x02000000      ; p1.25, 1-0 tells processor to take data from bus
19 LCD_RW      EQU      0x00400000      ; P0.22, 0 is writing to LCD, 1 is querying LCD
20
21            STMFD    sp!, {r0-r2,lr}
22
23            MOV      r2,r0              ;character is in r0
24
25            ;rs = 1, rw = 0, and e = 0
26            LDR      r1, =IOIPIN
27            LDR      r0, [r1]
28            BIC      r0, #LCD_E
29            ORR      r0, #LCD_RS
30            STR      r0, [r1]
31
32            LDR      r1, =IOOPIN
33            LDR      r0, [r1]
34            BIC      r0, #LCD_RW
35            STR      r0, [r1]
36
37            ;wait at least 6us
38            MOV      r0, #20
39            BL       Wait_10us          ;wait
40
41            ;e = 1
42            LDR      r1, =IOIPIN
43            LDR      r0, [r1]
44            ORR      r0, #LCD_E
45            STR      r0, [r1]
46
47            ;wait at least 6us
48            MOV      r0, #20
49            BL       Wait_10us          ;wait
50
```

```

51      ;write command to memory
52      LDR        r1, =IO1PIN
53      LDR        r0, [r1]
54      BIC        r0, #LCD_DATA    ;clear old bits 23-16
55      ORR        r0, r2, LSL #16
56      STR        r0, [r1]        ;write new command
57
58      ;wait at least 6us
59      MOV        r0, #20
60      BL         Wait_10us        ;wait
61
62      ;e = 0
63      LDR        r1, =IO1PIN
64      LDR        r0, [r1]
65      BIC        r0, #LCD_E
66      STR        r0, [r1]
67
68      ;wait at least 40us
69      MOV        r0, #80
70      BL         Wait_10us        ;wait
71
72      LDMFD      sp!, {r0-r2,pc}
73
74      END

```

LCD_string

```
1 ;Joshua Rothe, joshrothe@gmail.com
2 ;Narek Abrahamian, narek.abrahamian.287@my.csun.edu
3 ;Lab 9 LCD_string, Ver 0.02, last modified 5 Nov 2015
4 ;displays a string of LCD chars
5         GLOBAL LCD_string
6         area      mycode, CODE, Readonly
7
8 LCD_string
9         IMPORT LCD_cmd
10        IMPORT LCD_char
11
12        STMFD     sp!, {r0-r3,lr}
13
14        ;r0 will hold the character to display
15        ;r1 will hold the location to display (line)
16        MOV       r3, r0 ;temp holder
17
18        CMP       r1, #1 ;checks for second line indicator
19        BEQ       secondLine
20
21 firstLine
22        MOV       r2, #0x80 ;first line
23        B         main
24
25 secondLine
26        MOV       r2, #0xC0 ;second line
27        B         main
28
29 main
30        MOV       r0, r2
31        BL        LCD_cmd
32
33 loop    LDRB      r0,[r3],#1 ;get first char, r3 is copy of address of char
34        CMP       r0,#"_" ;check if ending char
35        BEQ       fin
36        BL        LCD_char ;send char to LCD
37        B         loop
38
39 fin     LDMFD     sp!, {r0-r3,pc}
40
41        END
```


Wait_10us

```
1 ;Joshua Rothe, joshrothe@gmail.com
2 ;Narek Abrahamian, narek.abrahamian.287@my.csun.edu
3 ;Lab 9 Wait_10us, Ver 0.02, last modified 5 Nov 2015
4 ;delay subroutine
5
6         GLOBAL Wait_10us
7         AREA reset, CODE, Readonly
8
9 Wait_10us
10
11 delay
12         MOV     r0, r0
13         SUBS    r0, r0, #1      ;deincrement loop counter
14         BNE     delay
15         BX      lr              ;return to program
16         END
```

Results

The code executed as expected, with some difficulty for task 1 and 2 as we had trouble getting the proper commands to execute in LCD_string. Once we were able to get the subroutines to execute properly (enough time in the delays are essential for this), the task of looping the code wasn't much more difficult, since most of the work was already complete; we merely needed to program the shifts to give the text its scrolling property.

Conclusion

Displaying text is a useful function, and knowing how to properly organize subroutines and initialize the systems required for the LCD display is essential for this. This lab was excellent practice in nesting subroutines and utilizing character strings and variable delay values.